



TALLER DE PROYECTO II. 2026

Investigación de Modelos: Cinemática Inversa y Detección de Autocolisiones

Grupo J1

Tarifa, Armando Ezequiel¹ (02893/4)

Guitera Woida, Emma¹ (03352/2)

Terruzzi, Ana Justina¹ (02557/9)

02/10/2026

¹{ezequiel.tarifa.2002,emma.guitera.woida,justina.terruzzi}@gmail.com

Índice

1. Introducción	2
2. Modelos implementados heredados	2
2.1. Cinemática Directa (FK) - Implementada en 2025	2
2.2. Cinemática Inversa (IK) - No implementada en 2025	2
2.3. Detección de Autocolisiones - Implementada en 2025	2
3. Cinemática Inversa Iterativa - Hito 1.2	3
3.1. Definición del problema	3
3.2. Algoritmos evaluados	3
3.2.1. Jacobiano transpuesto	3
3.2.2. Damped Least Squares (DLS)	3
3.2.3. CCD (Cyclic Coordinate Descent)	3
3.3. FABRIK (Forward And Backward Reaching Inverse Kinematics)	3
3.4. Tabla comparativa	3
3.5. Decisión: FABRIK	4
3.6. Formalización matemática del algoritmo	4
3.6.1. Paso 0 — Verificación de alcanzabilidad	4
3.6.2. Paso 1 — Fase Forward (llevar la punta al objetivo)	4
3.6.3. Paso 2 — Fase Backward (anclar la base)	4
3.6.4. Parámetros elegidos	5
3.7. Recuperación de ángulos articulares	5
4. Modelo de Esferas para Detección de Autocolisiones — Hito 1.3	6
4.1. Contexto: el modelo del año anterior	6
4.2. Adopción del mismo modelo	6
4.3. Fundamentación matemática	6
4.4. Distribución de esferas por eslabón	6
4.5. Matriz booleana de exclusión 5×5	6
4.5.1. Reglas de exclusión	7
4.5.2. Exclusión de articulación compartida	7
4.6. Colisión contra el plano base	7
5. Conclusión	8
Referencias	9

1. Introducción

En este documento de investigación se recopila la fundamentación teórica, comparación de algoritmos y decisiones de diseño correspondientes a los hitos **H1.2** (Cinemática inversa iterativa) y **H1.3** (Modelo de esferas para detección de autocolisiones) del proyecto J1.

A lo largo del informe se referencia lo que el grupo J5 del año 2025 implementó en su proyecto [1], para establecer qué se hereda directamente, qué se modifica y qué es nuevo en nuestro proyecto.

2. Modelos implementados heredados

Antes de avanzar con modelos nuevos, mencionaremos los modelos implementados por el grupo anterior [1].

2.1. Cinemática Directa (FK) - Implementada en 2025

El grupo J5 adoptó la convención **Denavit-Hartenberg (DH)** como marco teórico para modelar la cinemática directa. La implementación en C resultante construye cada matriz T_{i-1}^i combinando una rotación de corrección de montaje (R_{fix}), la rotación del servo y la traslación del eslabón, lo que es equivalente al producto DH estándar.

La cadena cinemática del brazo de 5 grados de libertad se modela como:

$$T_{\text{total}} = T_0^1 \cdot T_1^2 \cdot T_2^3 \cdot T_3^4 \cdot T_4^5$$

La secuencia de ejes de rotación es **Z-Y-Y-X-Y** para los servos 0 a 4. Cada R_{fix} alinea el eje físico del servo con el eje Z matemático para que todas las rotaciones se expresen uniformemente.

Esta implementación fue codificada en C (`cinematica.c`, `algebra_lineal.c`, `numpy.c`) para el ESP32 y replicada en Python (`cinematica.py`, `algebra_lineal.py`) para simulación.

Se hereda para el proyecto 2026 sin modificaciones en la lógica de cinemática directa.

2.2. Cinemática Inversa (IK) - No implementada en 2025

El proyecto del 2025 **no implementó cinemática inversa**. El control del brazo se realizaba exclusivamente mediante un potenciómetro analógico y botones pulsadores: el operador seleccionaba un servo y lo rotaba manualmente. El sistema verificaba colisiones con cada pose resultante, pero no existía la posibilidad de enviar un destino (X, Y, Z) y que el sistema calculara automáticamente los ángulos necesarios.

La implementación de cinemática inversa se implementará en nuestro proyecto 2026.

2.3. Detección de Autocolisiones - Implementada en 2025

El grupo J5 implementó un sistema de detección de autocolisiones basado en **esferas delimitadoras** [1]. El modelo define 9 esferas distribuidas en los 5 eslabones con radio uniforme $r = 1.0$ cm, verificando pares de esferas filtrados por una matriz booleana 5x5 para evitar falsos positivos entre eslabones adyacentes.

Se hereda para el proyecto 2026.

3. Cinemática Inversa Iterativa - Hito 1.2

3.1. Definición del problema

Dado un punto objetivo $\mathbf{p}^* \in \mathbb{R}^3$ en el espacio de trabajo del brazo, el problema de la cinemática inversa consiste en encontrar el vector de ángulos articulares:

$$\boldsymbol{\theta}^* = (\theta_0^*, \theta_1^*, \theta_2^*, \theta_3^*, \theta_4^*)$$

tal que la evaluación de la cinemática directa resulte en dicho punto:

$$\text{FK}(\boldsymbol{\theta}^*) = \mathbf{p}^*$$

Para un brazo de 5 grados de libertad con geometría no esférica, la resolución analítica cerrada es altamente compleja y propensa a errores de implementación debido a la cantidad de casos y condiciones. Por esta razón, se opta por un enfoque numérico iterativo capaces de ejecutarse eficientemente en el ESP32.

3.2. Algoritmos evaluados

Se estudiaron y compararon cuatro métodos de cinemática inversa iterativa:

3.2.1. Jacobiano transpuesto

Calcula el Jacobiano $J(\boldsymbol{\theta}) \in \mathbb{R}^{3 \times n}$ y actualiza los ángulos con:

$$\Delta\boldsymbol{\theta} = \alpha \cdot J^T \cdot (\mathbf{p}^* - \text{FK}(\boldsymbol{\theta}))$$

- **Ventajas:** No requiere inversión de matrices.
- **Desventajas:** Convergencia lenta (20-100 iteraciones), oscilaciones cerca de singularidades cinemáticas, requiere elegir α adecuadamente.

3.2.2. Damped Least Squares (DLS)

Resuelve el sistema regularizado:

$$\Delta\boldsymbol{\theta} = (J^T J + \lambda^2 I)^{-1} J^T \cdot (\mathbf{p}^* - \text{FK}(\boldsymbol{\theta}))$$

- **Ventajas:** Robusto frente a singularidades gracias al parámetro de amortiguación λ .
- **Desventajas:** Requiere una matriz $n \times n$ en cada iteración, costoso para el ESP32, el parámetro λ debe ser ajustado dinámicamente.

3.2.3. CCD (Cyclic Coordinate Descent)

Ajusta un ángulo por vez, recorriendo las articulaciones cíclicamente. Cada articulación i se rota para minimizar la distancia entre el efector y el objetivo en su plano de acción.

- **Ventajas:** Sin matrices, simple de implementar.
- **Desventajas:** Convergencia errática, no garantiza el camino más corto.

3.3. FABRIK (Forward And Backward Reaching Inverse Kinematics)

FABRIK es un algoritmo que resuelve la cinemática inversa de forma iterativa, ajustando sucesivamente los ángulos de las articulaciones para acercar el efector al objetivo. Opera con posiciones cartesianas de las articulaciones [2], [3].

- **Ventajas:** Sin matrices ni derivadas, convergencia en 5-20 iteraciones, movimientos naturales.
- **Desventajas:** No maneja restricciones angulares de forma nativa (se aplican como post-proceso).

3.4. Tabla comparativa

Criterio	Jac. Trans- puesto	DLS	FABRIK	CCD
Necesita matriz	No	Sí ($n \times n$)	No	No
Singularidades	Problemático	Robusto (λ)	Inmune	Robusto
Costo por iteración	Bajo	Medio-Alto	Muy bajo	Muy bajo

Convergencia típica	20–100 iter.	10–50 iter.	5–20 iter.	10–50 iter.
Naturalidad del movimiento	Media	Media	Alta	Baja
Implementación en C para ESP32	Media	Alta (inversión)	Baja-Media	Baja
Adecuado para ESP32	Aceptable	Aceptable	Óptimo	Aceptable

3.5. Decisión: FABRIK

Se selecciona **FABRIK** como algoritmo de cinemática inversa para el proyecto 2026. La decisión fue tomada en base a los siguientes items:

- **Sin inversión matricial:** Opera con sumas, restas y normalizaciones de vectores 3D. No necesita calcular jacobianos ni inversiones de matrices. Por lo tanto, su implementación en C es mas simple y rapida que las otras alternativas.
- **Inmunidad a singularidades:** Al no depender de derivadas, no existe configuración del brazo que genere una matriz singular.
- **Convergencia rápida:** Aristidou & Lasenby reportan convergencia 3-5x más rápida que métodos de Jacobiano para precisión equivalente [2].
- **Implementación real disponible:** Existe una librería de referencia (NocKinematics) optimizada específicamente para Arduino/ESP32 [4].
- **Movimiento natural:** Los trazados son suaves y predecibles.

3.6. Formalización matemática del algoritmo

Sean $\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_n$ las posiciones globales de las $n + 1$ articulaciones, con $\mathbf{p}_0$ siendo la base fija del brazo y $\mathbf{p}_n$ el extremo efector. Sean $l_i = \|\mathbf{p}_i - \mathbf{p}_{i-1}\|$ las longitudes mecánicas constantes de cada eslabón.

3.6.1. Paso 0 — Verificación de alcanzabilidad

Antes de iterar, se comprueba si el objetivo está dentro del rango máximo del brazo:

$$\|\mathbf{p}^* - \mathbf{p}_0\| \leq \sum_{i=1}^n l_i$$

Si no se cumple, el objetivo es inalcanzable. El sistema responde con error HTTP al Gemelo Digital sin ejecutar ninguna iteración.

3.6.2. Paso 1 — Fase Forward (llevar la punta al objetivo)

Se coloca el efector directamente en el objetivo y se propagan las restricciones de longitud hacia la base:

$$\begin{aligned} \mathbf{p}_n &\leftarrow \mathbf{p}^* \\ \mathbf{p}_{i-1} &\leftarrow \mathbf{p}_i + \frac{\mathbf{p}_{i-1} - \mathbf{p}_i}{\|\mathbf{p}_{i-1} - \mathbf{p}_i\|} \cdot l_i \quad \text{para } i = n, n-1, \dots, 1 \end{aligned}$$

3.6.3. Paso 2 — Fase Backward (anclar la base)

La base se ha “desplazado”, por lo que se reancla en el origen físico y se propagan las restricciones hacia adelante:

$$\begin{aligned} \mathbf{p}_0 &\leftarrow \mathbf{p}_{\text{base}} \\ \mathbf{p}_i &\leftarrow \mathbf{p}_{i-1} + \frac{\mathbf{p}_i - \mathbf{p}_{i-1}}{\|\mathbf{p}_i - \mathbf{p}_{i-1}\|} \cdot l_i \quad \text{para } i = 1, 2, \dots, n \end{aligned}$$

Los pasos 1 y 2 se repiten alternadamente hasta que:

$$\|\mathbf{p}_n - \mathbf{p}^*\| < \varepsilon \quad \text{o} \quad k > k_{\text{max}}$$

3.6.4. Parámetros elegidos

Parámetro	Valor	Justificación
ε (tolerancia)	0.5 cm	Resolución mecánica del SG90 ($1^\circ \approx 0.3$ cm en el extremo)
$k_{\max}$ (iteraciones)	20	Se reportan convergencia en 5–15 para cadenas similares

3.7. Recuperación de ángulos articulares

FABRIK determina posiciones cartesianas $\mathbf{p}_i$, pero los servos necesitan ángulos. La recuperación se realiza mediante:

$$\theta_i = \text{atan2}(\|\mathbf{d}_i \times \mathbf{z}_i\|, \mathbf{d}_i \cdot \mathbf{z}_i)$$

donde $\mathbf{d}_i = \mathbf{p}_{i+1} - \mathbf{p}_i$ es el vector director del eslabón y $\mathbf{z}_i$ es el eje de rotación del servo i expresado en coordenadas globales (obtenido de la cadena FK). El signo correcto se determina con la dirección del producto cruzado $\mathbf{z}_i \times \mathbf{d}_i$.

Tras el cálculo, cada ángulo se limita a los rangos mecánicos de los servos SG90:

Servo	Articulación	Mín (°)	Máx (°)
0	Base	0	170
1	Hombro	0	135
2	Codo	−170	0
3	Muñeca	—	—
4	Pinza	0	180

Si algún ángulo calculado queda fuera del rango permitido tras el clampeo, la pose se declara inválida.

4. Modelo de Esferas para Detección de Autocolisiones — Hito 1.3

4.1. Contexto: el modelo del año anterior

El grupo j5 (2025) diseñó e implementó un sistema de detección de autocolisiones basado en **esferas delimitadoras** [1]. El modelo demostró ser funcional durante las pruebas del prototipo 2025: detectaba correctamente colisiones entre eslabones no abyacentes y contra el plano base, bloqueando movimientos peligrosos.

4.2. Adopción del mismo modelo

Se evaluaron otras alternativas, como **Cápsulas** o **AABB / OBB**. Sin embargo, se concluyó que las esferas delimitadoras son la mejor opción para nuestro prototipo por las siguientes razones:

- **Ya está validado:** El grupo J5 lo probó extensivamente con el hardware real [1].
- **Costo computacional bajo:** La evaluación de colisión entre dos esferas se reduce a una sola operación de distancia euclidiana.
- **Compatibilidad directa:** El código C heredado ya implementa este modelo. No hay costo de re-implementación.
- **Suficiente para el caso de uso:** Los eslabones del brazo SG90 son cilindros cortos; las esferas proporcionan un margen de seguridad adecuado sin precisar geometrías mas complejas.

Las cápsulas (Swept Sphere Volumes) serían una mejora futura si se detectan falsos negativos con el hardware real, por el momento se mantendrá el modelo de esferas.

4.3. Fundamentación matemática

La intersección entre dos esferas S_A y S_B en $\mathbb{R}^3$ se evalúa con una única desigualdad euclidiana [5]:

$$d(\mathbf{c}_A, \mathbf{c}_B) = \sqrt{(c_A^x - c_B^x)^2 + (c_A^y - c_B^y)^2 + (c_A^z - c_B^z)^2} \leq r_A + r_B + \delta$$

Para el prototipo, se usa radio uniforme $r_{\text{esf}} = 1.0$ cm (heredado del año anterior) y tolerancia numérica $\delta = 10^{-9}$. La condición de colisión se simplifica a:

$$d(\mathbf{c}_A, \mathbf{c}_B) \leq 2 \cdot r_{\text{esf}}$$

4.4. Distribución de esferas por eslabón

Las coordenadas se definen en el espacio local de cada eslabón como vectores homogéneos $\tilde{\mathbf{c}}_{i,j}^{\text{loc}} = (x, y, z, 1)^T$. La transformación a coordenadas globales reutiliza la misma cadena de matrices $M_{\text{eslabon}}^{(i,j)}$ calculada por la FK:

$$\mathbf{c}_{i,j}^{\text{glob}} = M_{\text{eslabon}}^{(i)} \cdot \tilde{\mathbf{c}}_{i,j}^{\text{loc}}$$

Eslabón	Articulación	Centros locales (x, y, z)	Cant.
0	Base	(0, 0, 0)	1
1	Hombro	(0, 0, 0); (2, 0, 0)	2
2	Codo	(0, 0, 0); (2, 0.5, 0); (4, 0.5, 0)	3
3	Muñeca	(0, 0, 0)	1
4	Pinza	(0, 0, 0); (0, -2, 0)	2
	Total		9 esferas

Estos valores son los documentados en `configuracion.c` del prototipo 2025 [1]. Serán validados contra el hardware físico real una vez ensamblado, ajustando posiciones y radio si corresponde.

4.5. Matriz booleana de exclusión 5×5

Sin optimización, verificar colisiones entre las 9 esferas requeriría $\binom{9}{2} = 36$ evaluaciones de distancia en cada ciclo. La **Matriz de Colisiones** booleana (heredada de [1]) filtra los pares físicamente imposibles:

$$M_{\text{colisiones}} = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{pmatrix}$$

$M[i, j] = 1 \rightarrow$ verifica colisión entre eslabón i y eslabón j .

4.5.1. Reglas de exclusión

- **Diagonal** ($i = j$): Un eslabón no colisiona consigo mismo.
- **Adyacentes** ($|i - j| = 1$): Los eslabones consecutivos comparten una articulación física. Sus esferas extremas se superponen por diseño mecánico. Es físicamente imposible que estos eslabones colisionen entre sí.
- **Casi-adyacentes** ($|i - j| = 2$): La geometría del brazo y los límites angulares de los SG90 hacen que estos pares rara vez puedan alcanzar una configuración de colisión real.

4.5.2. Exclusión de articulación compartida

Para pares adyacentes que sí se verifican según la matriz, se excluye adicionalmente el cruce entre la última esfera del eslabón i y la primera del eslabón j , ya que comparten el punto de articulación:

Excluir: $(c_{i, \text{última}}, c_{j, \text{primera}})$

4.6. Colisión contra el plano base

Adicionalmente, cada esfera (excepto las del eslabón 0, que es la base) se verifica contra el plano del suelo ($Z = 0$):

$$c_{i,j}^{\text{glob}}[z] \leq r_{\text{esf}} \quad \forall i \in \{1, 2, 3, 4\}, \quad \forall j$$

Si cualquier esfera penetra el plano, la pose se rechaza inmediatamente.

5. Conclusión

La investigación establece que:

- 1) El modelo de **cinematica directa** (FK) del año anterior es matemáticamente correcto y se adopta sin modificaciones.
- 2) El modelo de **esferas delimitadoras** para colisiones fue probado en hardware real por el grupo J5 y es el más adecuado para las restricciones del ESP32. Se adopta directamente.
- 3) **FABRIK** es el algoritmo óptimo para IK en nuestro contexto: sin matrices, convergencia rápida, implementación simple en C. Es la contribución técnica nueva del grupo J1.

Referencias

- [1] Grupo J5 — Taller de Proyecto II 2025, «Brazo Robótico Seguro — Bitácora y documentación técnica». [En línea]. Disponible en: <https://github.com/tpII/2025-J5-Brazo-Robotico-Seguro/wiki>
- [2] A. Aristidou, «FABRIK: A fast, iterative solver for the Inverse Kinematics problem». [En línea]. Disponible en: <https://www.andreasaristidou.com/FABRIK.html>
- [3] Wikipedia, «Inverse Kinematics — Heuristic Methods». [En línea]. Disponible en: https://en.wikipedia.org/wiki/Inverse_kinematics#Heuristic_methods
- [4] Nocturnaled Community, «NocKinematics — FABRIK library for Arduino/ESP32». [En línea]. Disponible en: <https://github.com/Nocturnaled-Community/NocKinematics>
- [5] Mozilla Developer Network, «3D collision detection». [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Games/Techniques/3D_collision_detection